JURONG PIONEER JUNIOR COLLEGE

JC2 Theory Practice Set 2 (2026)

COMPUTING 9569/01 Paper 1 (Written)

SUGGESTED MARK SCHEME

General marking notes

- One mark is awarded for each marking point unless stated otherwise. Do not award more than the maximum for any part.
- Accept any correct and unambiguous pseudo-code, including Python-style code, unless the question asks for a specific form. Minor syntax slips should not be penalised if the logic is clear.
- Credit valid alternative answers that are correct, relevant to the context and at the same depth as the listed points.
- Where the question refers to the scenario, a generic answer earns the identification mark only.
- Table descriptions: solid underline = primary key, dashed underline = foreign key, double underline = part of the primary key and also a foreign key.

Paper blueprint

Q	Context	Main syllabus learning outcomes	Marks
1	CrowdTaskSG participant ID (GovTech)	2.4.1, 2.4.2 validation, verification, check digit; 3.2.2 ASCII in programs; 2.4.4 test data	13
2	OneMap nearby amenities and address search (SLA)	1.2.1 to 1.2.5 quicksort, insertion sort, binary search, hash table search, Big-O	13
3	PlayTime activity listings (OGP)	3.3.2 to 3.3.8 1NF to 3NF, keys, ER diagram, SQL query with joins	14
4	Singpass passkey (GovTech)	4.3.3 encryption, digital signature, authentication; 3.3.10 protecting data	10
5	OneMap barrier-free routing (SLA)	2.2.1 2D arrays; 2.2.5 to 2.2.7 recursion; 1.1.3 iteration	12
6	Spot the Scam! game (CATOS, SIT)	2.5.1 to 2.5.5 class diagram, inheritance, overriding, polymorphism	11
7	ReadSG booth and FormSG (NLB, CSA)	4.1.1 to 4.1.5 client-server, DNS, protocols; 4.2.1 web vs native; 4.3.1, 4.3.2 DoS, malware, firewall, IPS	12
8	AI-built clones of Adobe software (news, Oct 2026)	4.3.3 digital signatures; 3.3.12 version control; 2.4.4 testing; 3.4.1 to 3.4.3 social, ethical, legal, economic issues	15
		Total	**100**

Sources of contexts. Questions 1 to 7 use booth descriptions from the IMDA Digital for Life Festival 2026 Smart Nation showcase slides (pages 30 to 75: CATOS, CSA, GovTech Singpass Passkey and CrowdTaskSG, NLB ReadSG x S.U.R.E., MHA, SPF, OGP PlayTime, SLA OneMap). Question 8 uses October 2026 reports on generative AI being used to build free clones of Adobe Creative Cloud applications (Australian Financial Review, 16 October 2026; PhotoWorkout, 8 October 2026; Digital Camera World, 13 October 2026). The check-letter scheme in Q1, the database in Q3, the grid model in Q5 and the red-flag rules in Q6 are simplified models written for this paper and do not describe how the real systems work.

As with the revised Theory Practice Set 1, topics tested in the 2026 A-level Paper 2 (linear queue and stack, file handling, SQL table creation, array-based binary search tree) and the free space list are avoided.

Question 1: Validation, verification and check digits (CrowdTaskSG)

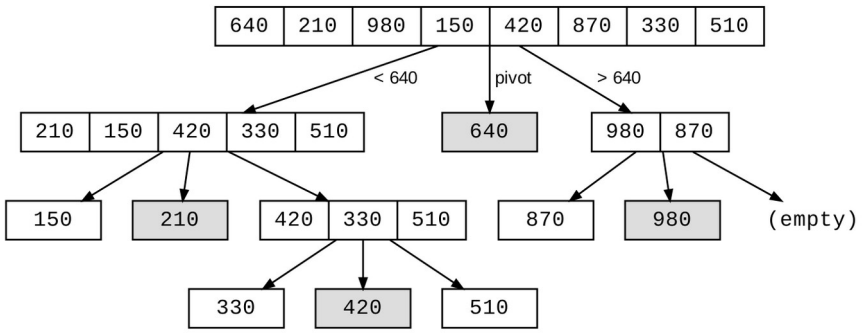
Part	Suggested answer and marking points	Marks
(a)	Validation is an automatic check by the program that data is sensible and follows set rules; it cannot tell whether the data is actually correct. Example: a range check that the age is 15 or above, or a format check on the participant ID. [1] Verification checks that the data entered matches the original source or what the user intended. Example: asking the participant to enter the email address twice, or retrieving name and date of birth from Singpass for the participant to confirm. [1] *Both definitions needed, each with a suitable example.*	**2**
(b)	5×6 + 7×5 + 3×4 + 8×3 + 1×2 = 30 + 35 + 12 + 24 + 2 = 103 [1] 103 MOD 11 = 103 − 99 = 4 [1] Position 4 of "ABCDEFGHJKL" is "E", so the ID is C57381E [1]	**3**
(c)	The digits 5 and 7 are multiplied by different weights, so swapping them changes the weighted total: 7×6 + 5×5 + 3×4 + 8×3 + 1×2 = 105, 105 MOD 11 = 6, giving check letter "G". [1] The recalculated check letter "G" does not match the "E" that was typed, so the ID is rejected and the transposition error is detected. [1]	**2**

(d)	`FUNCTION IsValidID(ID : STRING) RETURNS BOOLEAN`
	`  DECLARE Letters : STRING`
	`  DECLARE Total, i, Code : INTEGER`
	`  Letters ← "ABCDEFGHJKL"`
	`  IF LENGTH(ID) <> 7 THEN`
	`  RETURN FALSE`
	`  ENDIF`
	`  IF ID[0] <> "C" THEN`
	`  RETURN FALSE`
	`  ENDIF`
	`  Total ← 0`
	`  FOR i ← 1 TO 5`
	`  Code ← ASC(ID[i])`
	`  IF Code < 48 OR Code > 57 THEN`
	`  RETURN FALSE  // not a digit`
	`  ENDIF`
	`  Total ← Total + (Code - 48) * (7 - i)  // weights 6, 5, 4, 3, 2`
	`  NEXT i`
	`  IF ID[6] = Letters[Total MOD 11] THEN`
	`  RETURN TRUE`
	`  ELSE`
	`  RETURN FALSE`
	`  ENDIF`
	`ENDFUNCTION`
	1 mark: length check (7 characters) and first character "C"
	1 mark: loop over positions 1 to 5, rejecting any character whose ASCII code is outside 48 to 57
	1 mark: converts each digit to its value (ASC(ch) − 48)
	1 mark: multiplies by the correct weight (6, 5, 4, 3, 2) and accumulates the total
	1 mark: compares ID[6] with the letter at position Total MOD 11 and returns the correct Boolean
	Checked by running an equivalent program: C40213J and C57381E valid; C75381E, C5738XE, C57381 and D40213J invalid.
		5

(e)	15: it is the boundary of the valid range (the lowest accepted age), so it should be	**1**

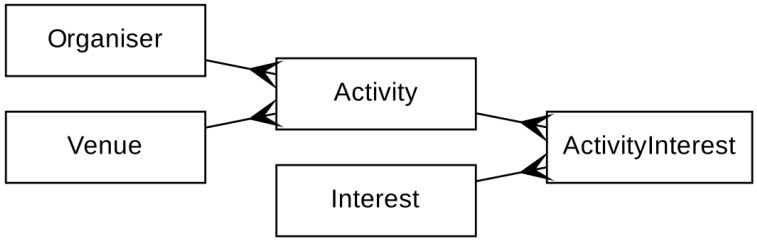
Part	Suggested answer and marking points	Marks
	accepted. (Accept 14 as the boundary value that should be rejected, if the expected outcome is stated.)	

Question 2: Sorting and searching (OneMap)

Part	Suggested answer and marking points	Marks
(a)	 Final sorted list: 150 210 330 420 510 640 870 980 1 mark: first partition with 640 as pivot: values less than 640 (210 150 420 330 510) on one side and greater (980 870) on the other 1 mark: sub-lists partitioned recursively with their first element as pivot (210, then 420; 980) 1 mark: correct final sorted list, with annotations identifying pivots and the recursion stopping at sub-lists of 0 or 1 element *In-place partition schemes give a different order inside each sub-list. Accept any valid partitioning with the first element as pivot.*	**3**
(b)	Pass 1: 210 640 980 150 420 870 330 510 [1] Pass 2: 210 640 980 150 420 870 330 510 (980 is already in place) [1] Pass 3: 150 210 640 980 420 870 330 510 [1]	**3**
(c)	The list is already sorted except for the new item. Insertion sort only has to shift the new item back into position, so it makes very few comparisons (close to its best case of O(n)). [1] Quicksort with the first element as pivot performs very badly on sorted or nearly sorted data: each partition removes only one element, giving its worst case of $O(n^2)$ and deep recursion. [1]	**2**
(d)(i)	17 (2^{17} = 131 072 ≥ 120 000, while 2^{16} = 65 536 is too few)	**1**
(d)(ii)	Linear search: O(n), so up to 120 000 comparisons in the worst case. [1] Binary search: O(log n), because each comparison halves the part of the array still to be searched, so at most 17 comparisons. [1]	**2**
(e)	The hash function calculates the index of a postal code directly from its value, so the item can usually be accessed in one step, O(1), without comparisons that halve the search space. [1] Disadvantage, any one: collisions slow the search down; the data is not kept in order, so it cannot easily list postal codes in order or find the nearest or a range of postal codes; extra empty space is needed in the table. [1]	**2**

Question 3: Databases (PlayTime)

Part	Suggested answer and marking points	Marks
(a)	A table in 1NF must have atomic (single) values in every field, with no repeating groups. [1] The Interests field holds more than one value in a field, for example "Nature,	**2**

Part	Suggested answer and marking points	Marks
	Outdoor" for A01 and "Reading, Art" for A03. [1]	
(b)	Organiser(OrganiserID, OrganiserName, OrganiserType) Venue(PostalCode, VenueName) Activity(ActivityID, Title, ActivityDate, MinAge, MaxAge, OrganiserID, PostalCode) Interest(InterestCode, InterestName) ActivityInterest(ActivityID, InterestCode) 1 mark: Organiser and Venue correct 1 mark: Activity with correct primary key and all its own attributes 1 mark: Activity with OrganiserID and PostalCode as foreign keys 1 mark: Interest with InterestCode as primary key 1 mark: ActivityInterest with composite primary key (ActivityID, InterestCode), both shown as foreign keys *VenueName depends on PostalCode, not on ActivityID, which is why Venue is a separate table (transitive dependency). Accept VenueID as the key of Venue if PostalCode is then an attribute of Venue.*	**5**
(c)	 1 mark: Organiser to Activity, one-to-many 1 mark: Venue to Activity, one-to-many 1 mark: Activity to ActivityInterest and Interest to ActivityInterest, both one-to-many *A direct many-to-many relationship between Activity and Interest scores 0 for the last mark.*	**3**

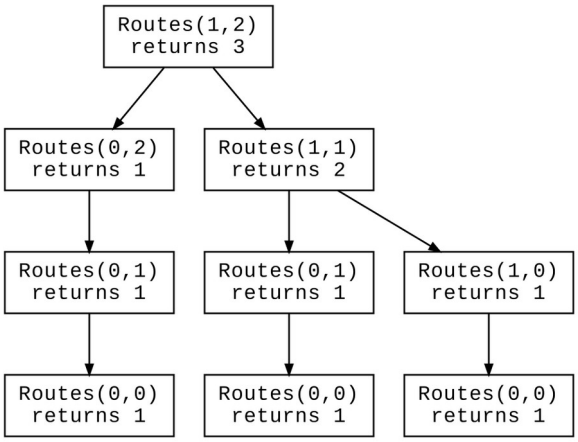
(d)	`SELECT A.Title, A.ActivityDate, V.VenueName`
	`FROM Activity AS A`
	`  INNER JOIN Venue AS V ON A.PostalCode = V.PostalCode`
	`  INNER JOIN ActivityInterest AS AI`
	`  ON A.ActivityID = AI.ActivityID`
	`  INNER JOIN Interest AS I ON AI.InterestCode = I.InterestCode`
	`WHERE I.InterestName = 'Outdoor'`
	`  AND A.MinAge <= 6 AND A.MaxAge >= 6`
	`ORDER BY A.ActivityDate ASC;`
	1 mark: SELECT Title, ActivityDate, VenueName
	1 mark: correct joins of Activity, Venue, ActivityInterest and Interest
	1 mark: WHERE InterestName = 'Outdoor' AND MinAge <= 6 AND MaxAge >= 6 (or 6 BETWEEN MinAge AND MaxAge)
	1 mark: ORDER BY ActivityDate ASC (ASC optional)
	Checked in SQLite with the sample rows: returns Junior Nature Walk (2026-11-07, Singapore Botanic Gardens) and Family Kite Day (2026-11-14, Marina Barrage).
		4

Question 4: Public key cryptography and authentication (Singpass passkey)

Part	Suggested answer and marking points	Marks
(a)	The public key can only be used to check (verify) signatures, not to create them, and the private key cannot be worked out from it, so a stolen copy of the server's stored public keys does not let anyone log in. [1]	**2**

Part	Suggested answer and marking points	Marks
	Anyone who has the private key can create valid signatures and log in as the user, so it must stay on the phone (in secure storage) and never be sent over the network. [1]	
(b)	The phone signs (encrypts) the challenge, or a hash of it, with the user's private key, after the user unlocks the passkey, and sends the signature back to the server. [1] The server uses the public key stored with the user's account to verify (decrypt) the signature and checks that it matches the challenge it sent. [1] If they match, only the holder of the private key could have produced the signature, so the user is authenticated. Because the challenge is new and random each time, a signature captured earlier cannot be replayed. [1]	**3**
(c)	With a password and OTP, the user types secrets that a fake website can capture and immediately use (relay) on the real Singpass site. [1] With a passkey, nothing secret is typed or sent: the private key never leaves the phone, and the signature only answers one challenge. [1] The passkey is tied to the legitimate Singpass domain, so the phone will not use it on a fake website with a different domain, so the phishing site gets nothing it can reuse. [1]	**3**
(d)	It combines two different types of factor: something the user has (the phone holding the private key) [1] and something the user is (fingerprint or face) or knows (the 6-digit passcode), needed to unlock the passkey. [1]	**2**

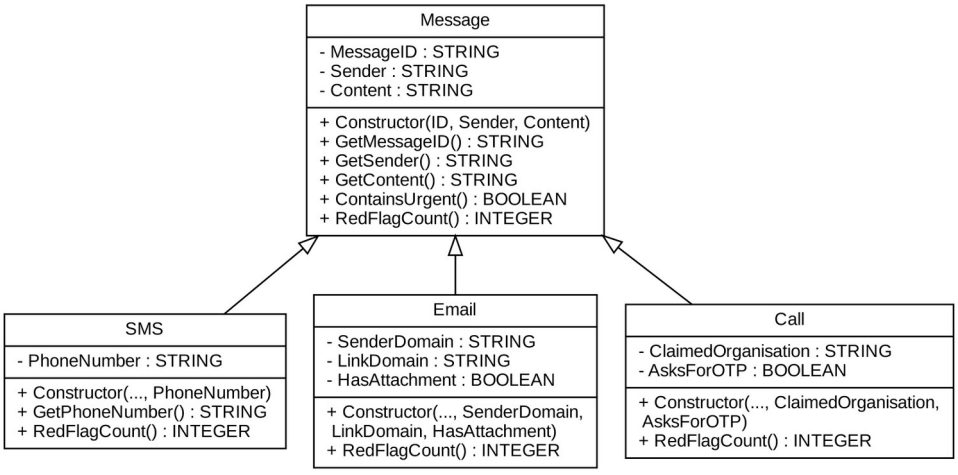
Question 5: Recursion and 2D arrays (OneMap barrier-free routing)

Part	Suggested answer and marking points	Marks
(a)	 1 mark: Routes(1,2) calls Routes(0,2) and Routes(1,1) 1 mark: all further calls shown: Routes(0,2) → (0,1) → (0,0); Routes(1,1) → (0,1) → (0,0) and (1,0) → (0,0) 1 mark: correct return values, with Routes(1,1) returning 2 and Routes(1,2) returning 3	**3**
(b)	3 *Routes: along row 0 to column 2, down to row 1, then either east then south or south then east (2 routes); down column 0 then along row 2 (1 route).*	**1**
(c)	Lines 03 to 05: if the square cannot be used (FALSE), there are no routes through it, so 0 is returned and that path is abandoned. [1] Lines 06 to 08: when the starting square [0][0] is reached, exactly one complete route has been found, so 1 is returned. This stops the recursion. [1]	**2**

Part	Suggested answer and marking points	Marks
(d)	Each call makes up to two further calls, so the number of calls grows exponentially with the size of the map. [1] The same squares are recalculated many times (for example, Routes(0,1) is calculated twice in part (a)), and very deep recursion may also cause a stack overflow. [1]	**2**

(e)	`FUNCTION RoutesIter(Map, R : INTEGER, C : INTEGER) RETURNS INTEGER`
	`  DECLARE Count : ARRAY[0:R, 0:C] OF INTEGER`
	`  FOR i ← 0 TO R`
	`  FOR j ← 0 TO C`
	`  IF Map[i][j] = FALSE THEN`
	`  Count[i][j] ← 0`
	`  ELSE`
	`  IF i = 0 AND j = 0 THEN`
	`  Count[i][j] ← 1`
	`  ELSE`
	`  Count[i][j] ← 0`
	`  IF i > 0 THEN`
	`  Count[i][j] ← Count[i][j] + Count[i - 1][j]`
	`  ENDIF`
	`  IF j > 0 THEN`
	`  Count[i][j] ← Count[i][j] + Count[i][j - 1]`
	`  ENDIF`
	`  ENDIF`
	`  ENDIF`
	`  NEXT j`
	`  NEXT i`
	`  RETURN Count[R][C]`
	`ENDFUNCTION`
	1 mark: nested loops over rows 0 to R and columns 0 to C, in an order that fills squares north and west before they are needed
	1 mark: Count[i][j] ← 0 for an unusable square and Count[0][0] ← 1
	1 mark: Count[i][j] = Count[i − 1][j] + Count[i][j − 1], with checks for i > 0 and j > 0
	1 mark: returns Count[R][C]
	Checked by running an equivalent program: returns 3 for MapA (1, 2) and 3 for MapB (2, 3).
		4

Question 6: Object-oriented programming (Spot the Scam!)

Part	Suggested answer and marking points	Marks
(a)	 Any five of the following, 1 mark each:	**5**

Part	Suggested answer and marking points	Marks
	• superclass Message with MessageID, Sender and Content • subclasses SMS (PhoneNumber), Email (SenderDomain, LinkDomain, HasAttachment) and Call (ClaimedOrganisation, AsksForOTP) • inheritance arrows from each subclass to Message • attributes private (-) and methods public (+) • constructor and getter methods in Message • RedFlagCount() in Message and overridden in each subclass *Do not award the second point if attributes of Message are repeated in the subclasses.*	

(b)	`CLASS Email INHERITS Message`
	`  PRIVATE SenderDomain : STRING`
	`  PRIVATE LinkDomain : STRING`
	`  PRIVATE HasAttachment : BOOLEAN`
	
	`  PUBLIC PROCEDURE NEW(ID, Sender, Content, SDom, LDom, Att)`
	`  SUPER.NEW(ID, Sender, Content)`
	`  SenderDomain ← SDomain`
	`  LinkDomain ← LDomain`
	`  HasAttachment ← Attach`
	`  ENDPROCEDURE`
	
	`  PUBLIC FUNCTION RedFlagCount() RETURNS INTEGER`
	`  DECLARE Count : INTEGER`
	`  Count ← SUPER.RedFlagCount()  // "urgent" flag`
	`  IF LinkDomain <> SenderDomain THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  IF HasAttachment = TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  RETURN Count`
	`  ENDFUNCTION`
	`ENDCLASS`
	1 mark: class header showing Email inherits from Message, with the three private attributes
	1 mark: constructor calls the superclass constructor with ID, sender and content, and sets the three Email attributes
	1 mark: RedFlagCount() starts from the value returned by the superclass's RedFlagCount()
	1 mark: adds 1 when the link domain differs from the sender domain and 1 when there is an attachment, and returns the count
		4

(c)	The scoring code calls RedFlagCount() on every message in the round, treating each one as a Message; at run time each object runs its own overridden version. [1] DeepfakeVideo can be written as a new subclass of Message that overrides RedFlagCount() with its own rules, so the existing scoring code works with it unchanged (no IF statements to test the type of each message). [1]	**2**

Question 7: Networks and network security (ReadSG booth, FormSG)

Part	Suggested answer and marking points	Marks
(a)	The client (the browser on the visitor's phone) displays the form and sends a request containing the visitor's answers to the server. [1] The server (FormSG) waits for requests from many clients, receives and	**2**

Part	Suggested answer and marking points	Marks
	processes the submission (for example, stores the answers), and sends a response, such as a confirmation page, back to the client. [1]	
(b)	The phone sends a query containing the domain name form.gov.sg to a DNS server (if the address is not already cached). [1] The DNS server looks up the name, asking other DNS servers in the hierarchy if necessary, and returns the IP address, which the phone then uses to connect to the FormSG server. [1]	**2**
(c)	Any two, 1 mark each: • visitors can open the form at once by scanning a QR code or following a link, with no need to download and install an app • it works on any phone with a browser, whatever its operating system, so one version serves all visitors • it uses no storage on the visitor's phone and does not need app permissions • changes to the form are made once on the server and every visitor sees the latest version immediately	**2**
(d)	Protocols are agreed rules for the format, order and timing of messages and for handling errors, so that devices and software from different manufacturers (the phone, routers and the server) can understand each other, for example HTTP, TCP and IP.	**1**
(e)	This is a distributed denial of service (DDoS) attack: the infected computers send so many requests that the server's processing power, memory or network bandwidth is used up, so genuine users' requests are delayed or not answered. [1] A firewall filters traffic using rules such as allowed IP addresses and ports; the attack traffic comes from thousands of different addresses and looks like normal web requests to an allowed port, so the rules cannot easily tell it apart from genuine traffic. [1] An IPS monitors traffic for suspicious patterns (for example, an abnormal rate of requests) and automatically blocks or drops the attacking traffic before it reaches the server. [1]	**3**
(f)	A virus attaches itself to another program or file and needs a user action, such as running the file, to spread. [1] A worm is a standalone program that spreads by itself across a network, by exploiting weaknesses, without any user action. [1]	**2**

Question 8: Generative AI, software clones and ethics

Part	Suggested answer and marking points	Marks
(a)	Risk: the download may be a fake or altered copy containing malware (for example, a trojan or ransomware) that could steal data or damage files. [1] The developer signs the file with their private key (a signed hash of the file); the user's computer uses the developer's public key, from a trusted digital certificate, to check the signature against a fresh hash of the downloaded file. [1] If the check succeeds, the file really came from that developer and has not been altered since it was signed; an altered or fake file is detected and can be rejected. [1] *A signature shows who published the file and that it is unchanged. It does not prove the software is free of bugs or harmful behaviour.*	**3**
(b)	Any three, 1 mark each: • version control keeps a record of every change, with who (which agent) made it and when, so a faulty change can be traced	**3**

Part	Suggested answer and marking points	Marks
	• earlier working versions can be restored (rolled back) if a change breaks the program • changes made at the same time to the same file can be detected and merged, so one agent does not overwrite another's work • naming conventions (for files, branches, versions, identifiers) make it clear which version is the latest and keep the code consistent and easy to understand for humans and agents	
(c)(i)	Tests can only check the cases that were tested. There may be inputs or combinations (for example, unusual file types or very large files) that were never tested, and 2 of the 309 test files already failed. Tests show the presence of errors, not their absence.	**1**
(c)(ii)	Normal: a valid image file of a supported type, for example a small layered PSD file. Expected: the file opens and its layers display correctly. [1] Abnormal: for example a text file renamed with a .psd extension, a corrupted image file, or an unsupported file type. Expected: the application rejects the file with a suitable error message and does not crash. [1]	**2**
(d)	Mark by levels. Indicative content (credit other relevant points): **Social** • free tools widen access for students and small creators who cannot afford subscriptions • jobs and skills of software developers change: fewer people are needed to write code, while skills in testing, reviewing and specifying grow **Ethical** • is it fair to copy the look and features of a product that another company spent years and large sums developing? • responsibility for AI-generated code: a computing professional who publishes it should test it, avoid harm to users, be honest about its limits (early version, incomplete features) and respect intellectual property, as in a code of conduct • quality and security risk: users may trust AI-built software that has not been properly checked **Legal** • copyright and intellectual property: copying the source code, logos or icons of a product would be infringement, while re-creating features independently may be legal, but the boundary is unclear when an AI may have learned from copyrighted material • trademarks: names or designs that are too similar may mislead users; open-source licences decide how the clones themselves may be reused **Economic** • lower cost of building software reduces the advantage of established subscription (SaaS) companies, which may lose customers and revenue and need to cut prices or add value (cloud sync, support, file compatibility) • more competition and new start-ups; investors may fear lower future profits for software companies Levels: 5 to 6 marks: a balanced discussion covering at least three of social, ethical, legal and economic issues, each explained in this context, with a reasoned conclusion 3 to 4 marks: several relevant issues explained, but limited balance or some areas missing 1 to 2 marks: issues listed with little explanation or not linked to the context	**6**

END OF MARK SCHEME